Name :- Divya Ajay Pandey
Prn:- 202501050016
Practical 1

1.1. Practice Lab Assignments
1.1.1. Calculate Momentum
1.1.2 Conditional Calculation based on the Number of Digits
1.1.3. Days Between Two Dates
1.1.4. Reverse a Number
1.1.5. Multiplication Table
1.2 Lab Assignments
1.2.1. Pass or Fail
1.2.2. Fibonacci Series using Recursive Function
1.2.3. Pattern-1
1.2.4. Pattern-2

Practical 01

1.1.1 calculate momentum

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum is calculated using the formula:

where:

is the mass of the object (in kilograms).

is the velocity of the object (in meters per second).

Input Format:

A single floating-point number representing the mass of the object in kilograms.
A single floating-point number representing the velocity of the object in meters per second.

Output Format:

The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

Code:

```
# Read m and v
m = float(input())
v = float(input())
# Calculate momentum and display
p = m * v
print(f"{p:.2f}kgm/s"
```

CODETANTRA

```
momentum = mass * velocity
print(f"
{momentum:.2f}kgm/s")
```

Explorer

Test case 1

50 ms

Expected output

5.0

10.0

50.00kgm/s

06:23

mitaoe.codetantra.com/

CODETANTRA

```
momentum = mass * velocity
print(f"
{momentum:.2f}kgm/s")
```

Explorer

Test case 2

54 ms

Expected output

1.1.2 Conditional Calculation based on the Number of Digits

Write a Python program that accepts an integer as input. Depending on the number of digits in n.

Constraints:

1 ≤ n ≤ 999

Input Format:

The input consists of a single integer n.

Output Format:

If n is a single-digit number, print its square.

If n is a two-digit number, print its square root (rounded to two decimal places).

If n is a three-digit number, print its cube root (rounded to two decimal places).

Else print "Invalid".

```
n = int(input())
# Check for "Invalid" boundaries first to simplify the rest of the logic
if n < 1 or n = 1000:
print("Invalid")
elif n < 10:
# We know n = 1 because the first check failed
print(n**2)
elif n < 100:
# We know n = 10 because the previous check failed
print(f"{n**0.5:.2f}")
else:
# We know n = 100 and n < 1000 because all previous checks failed
print(f"{n**(1/3):.2f}")
```

06:25

mitaoe.codetantra.com/

CODETANTRA

```
if 1 <= number <= 9:
    print(number ** 2)
elif 10 <= number <= 99:
    print(f"{math.sqrt(number):.2f}")
elif 100 <= number <= 999:
    print(f"{math.cbrt(number):.2f}")
else:
    print("Invalid")
```

Explorer

4 out of 4 shown test case(s) passed

3 out of 3 hidden test case(s) passed

06:26

mitaoe.codetantra.com/

CODETANTRA

```
if 1 <= number <= 9:
    print(number ** 2)
elif 10 <= number <= 99:
    print(f"{math.sqrt(number):.2f}")
elif 100 <= number <= 999:
    print(f"{math.cbrt(number):.2f}")
else:
    print("Invalid")
```

Explorer

Test case 1

42 ms

Expected output

06:26

mitaoe.codetantra.com/

CODETANTRA

```
elif 1 <= number <= 9:
    print(number ** 2)
elif 10 <= number <= 99:
    print(f"{math.sqrt(number):.2f}")
elif 100 <= number <= 999:
    print(f"{math.cbrt(number):.2f}")
else:
    print("Invalid")
```

Explorer

Test case 2

28 ms

06:26

mitaoe.codetantra.com/

CODETANTRA

```
if 1 <= number <= 9:
    print(number ** 2)
elif 10 <= number <= 99:
    print(f"{math.sqrt(number):.2f}")
elif 100 <= number <= 999:
    print(f"{math.cbrt(number):.2f}")
else:
    print("Invalid")
```

Explorer

Test case 3

34 ms

06:26

mitaoe.codetantra.com/

CODETANTRA

```
if 1 <= number <= 9:
    print(number ** 2)
elif 10 <= number <= 99:
    print(f"{math.sqrt(number):.2f}")
elif 100 <= number <= 999:
    print(f"{math.cbrt(number):.2f}")
else:
    print("Invalid")
```

Explorer

Test case 4

30 ms

Expected output

1.1.3. Days Between Two Dates

Write a Python program to calculate the number of days between two dates.

Input Format:

The first line contains the first date in the format YYYY–MM–DD.

The second line contains the second date in the format YYYY–MM–DD.

Output Format:

An integer representing the number of days between the given dates.

```
from datetime import date
def calculate_days(first_date_str, second_date_str):
# Convert the string inputs into date objects
# fromisoformat handles 'YYYY-MM-DD' automatically
d1 = date.fromisoformat(first_date_str)
d2 = date.fromisoformat(second_date_str)
# Calculate the difference between the dates
delta = d2 - d1
return delta
# Read the dates as standard input
date1_str = input().strip()
date2_str = input().strip()
# Print the absolute value of the days to ensure a positive integer
print(abs(calculate_days(date1_str, date2_str).days))
```

CODETANTRA

```
date1 = input().strip()
date2 = input().strip()

d1 = datetime.strptime(date1, "%Y-%m-%d")
d2 = datetime.strptime(date2, "%Y-%m-%d")

difference = d2 - d1
print(difference.days)
```

Average time
0.029 s
29.50 ms

Maximum time
0.031 s
31.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1
29 ms

Test case 1

29 ms

Expected output

2014-07-02

2014-11-02

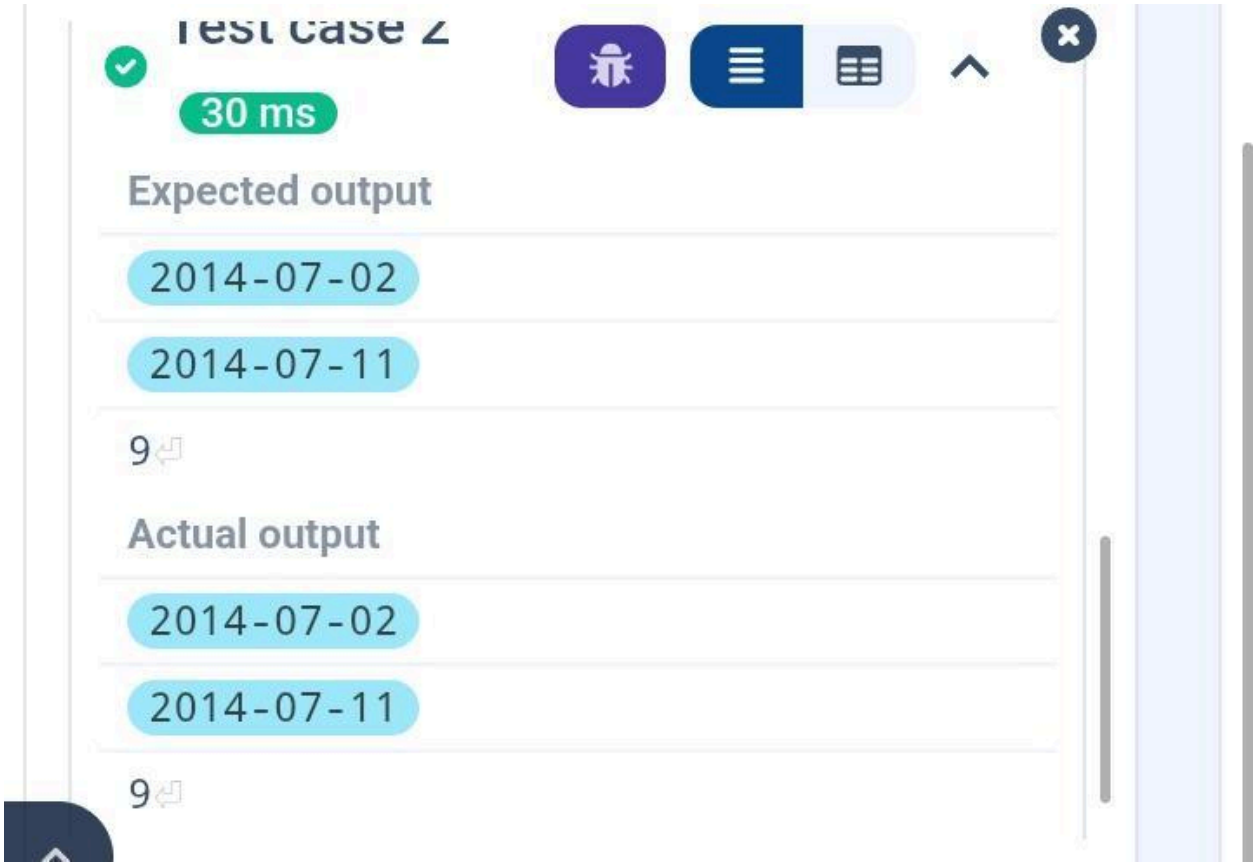
123

Actual output

2014-07-02

2014-11-02

123



1.1.4. Reverse a Number

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

● The input is a single integer.

Output Format:

● The output prints a single integer, which is the reversed number.

```
n = int(input())
reversed_num = 0
while n > 0:
digit = n % 10 # Get the last digit
reversed_num = reversed_num * 10 + digit # Add digit to the new
number
n = n / 10 # Remove the last digit from n
print(reversed_num)
```

Aliter: The most Pythonic Way —

```
# Read the input as an integer
num = int(input())
# Convert to string, reverse it with slicing [ :-1], and convert back to
int
reversed_num = int(str(num)[ :-1])
print(reversed_num)
```

3 out of 3 hidden test case(s) passed

Test case 1

15 ms

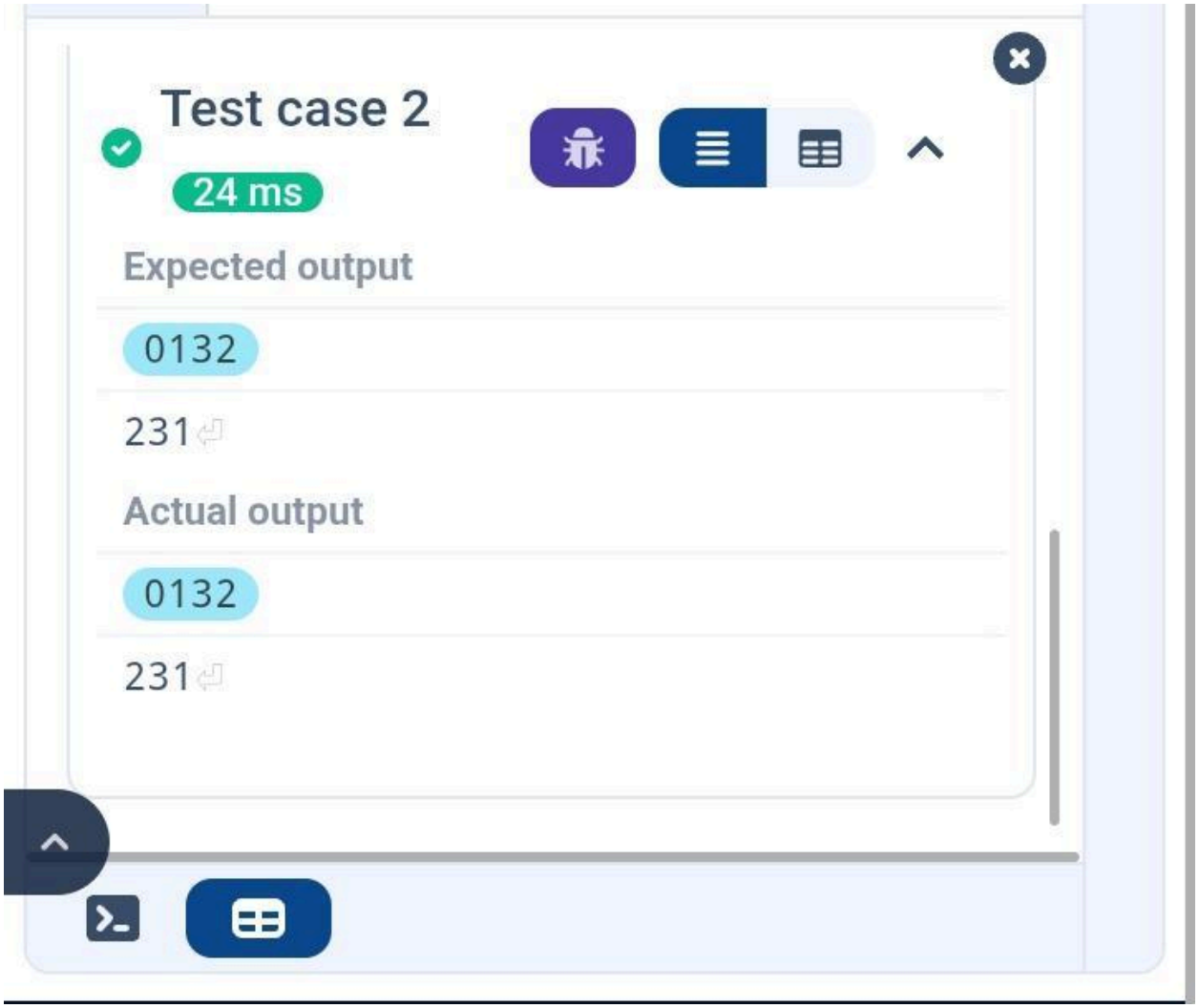
Expected output

5367

7635

Actual output

5367



1.1.5. Multiplication Table

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

● The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

● Print the multiplication table for the given number.

<number> x <multiplier> = <result>

Note:

● The character "x" is used in the output to represent multiplication.

● Refer to the visible test-cases for more clarity.

```
num = int(input())
for i in range(1, 11):
print(f"{num} x {i} = {num * i}")
```

Average time	Maximum time
0.029 s	**0.038 s**
29.00 ms	38.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 38 ms Debug

Expected output	Actual output
8	8
8 x 1 = 8	8 x 1 = 8
8 x 2 = 16	8 x 2 = 16
8 x 3 = 24	8 x 3 = 24

< Prev Reset Submit

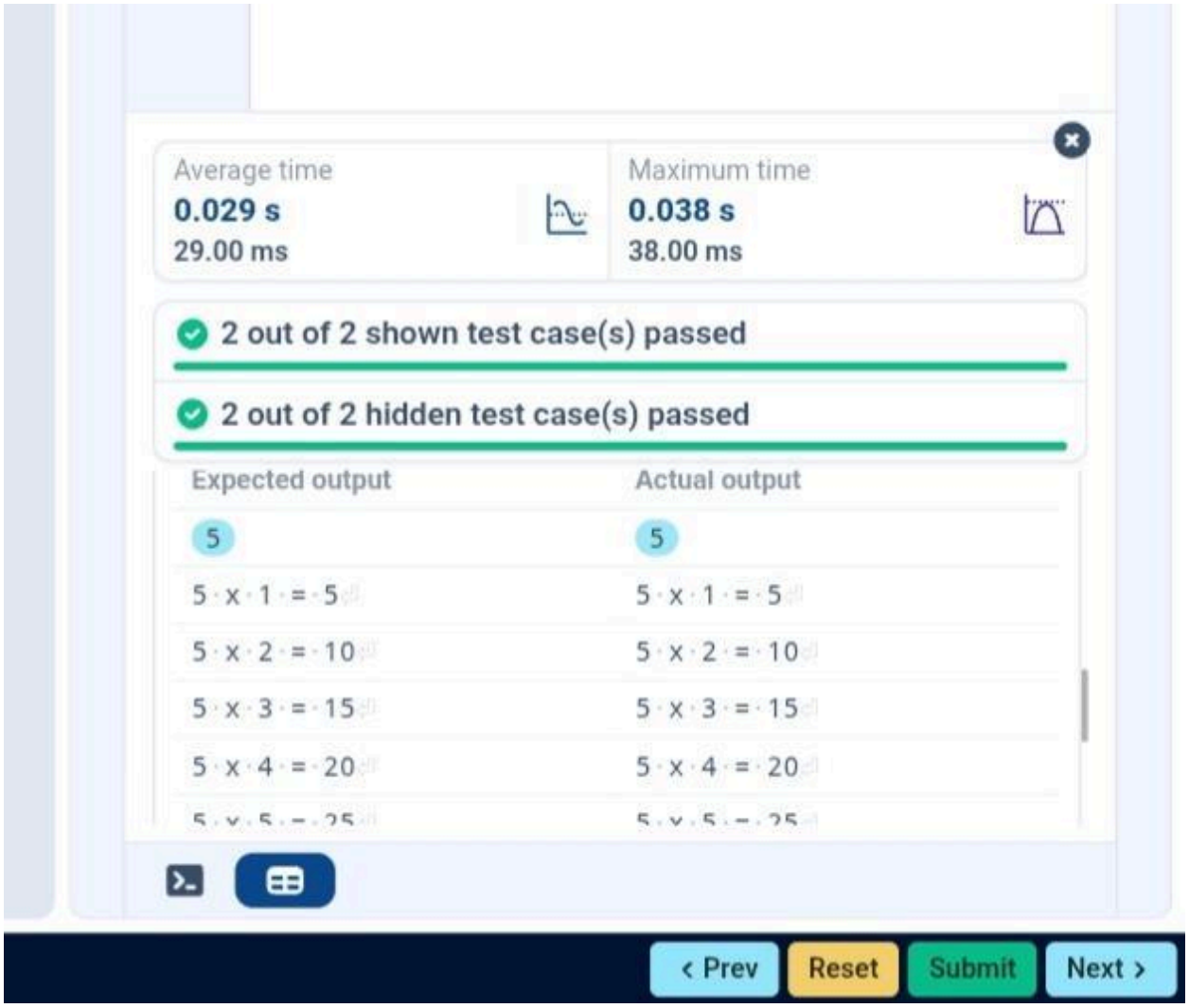
Average time	Maximum time
0.029 s	**0.038 s**
29.00 ms	38.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

8 x 3 = 24	8 x 3 = 24
8 x 4 = 32	8 x 4 = 32
8 x 5 = 40	8 x 5 = 40
8 x 6 = 48	8 x 6 = 48
8 x 7 = 56	8 x 7 = 56
8 x 8 = 64	8 x 8 = 64

‹ Prev Reset Submit Next ›



1.2 Lab Assignments

1.2.1. Pass or Fail

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

● If the aggregate percentage is greater than 75, the grade is Distinction.

● If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.

● If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.

● If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

● The first input will be an integer n, the number of courses.

● The second input will be n integers representing the marks of the student in each of the n courses, separated by a space.

Output Format:

If the student passes all courses:

● Print the aggregate percentage (formatted to two decimal places).

● Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

● "Fail".

Note:

● Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses.

```
# Read the number of courses
n = int(input())
# Read the marks as a list of integers
marks = list(map(int, input().split()))

# Check if the student failed any course (marks < 40)
if any(m < 40 for m in marks):
print("Fail")
else:
# Calculate aggregate percentage
aggregate_percentage = sum(marks) / n
# Determine the grade based on the percentage
if aggregate_percentage > 75:
grade = "Distinction"
elif 60 = aggregate_percentage < 75:
grade = "First Division"
elif 50 = aggregate_percentage < 60:
grade = "Second Division"
elif 40 = aggregate_percentage < 50:
grade = "Third Division"
# Note: Logic implies if percentage < 40 it would fail,
# but the initial check covers individual failures.
# Print the results
print(f"Aggregate Percentage: {aggregate_percentage:.2f}")
print(f"Grade: {grade}")
```

Test case 1

39 ms

Expected output	Actual output
4	4
55 65 78 89	55 65 78 89
Aggregate Percentage: 71.75	Aggregate Percentage: 71.75
Grade: First Division	Grade: First Division

< Prev Reset Submit Next >

Test case 2

44 ms

Expected output	Actual output
5	5
56 78 97 86 93	56 78 97 86 93
Aggregate Percentage: 82.00	Aggregate Percentage: 82.00
Grade: Distinction	Grade: Distinction

< Prev Reset Submit Next >

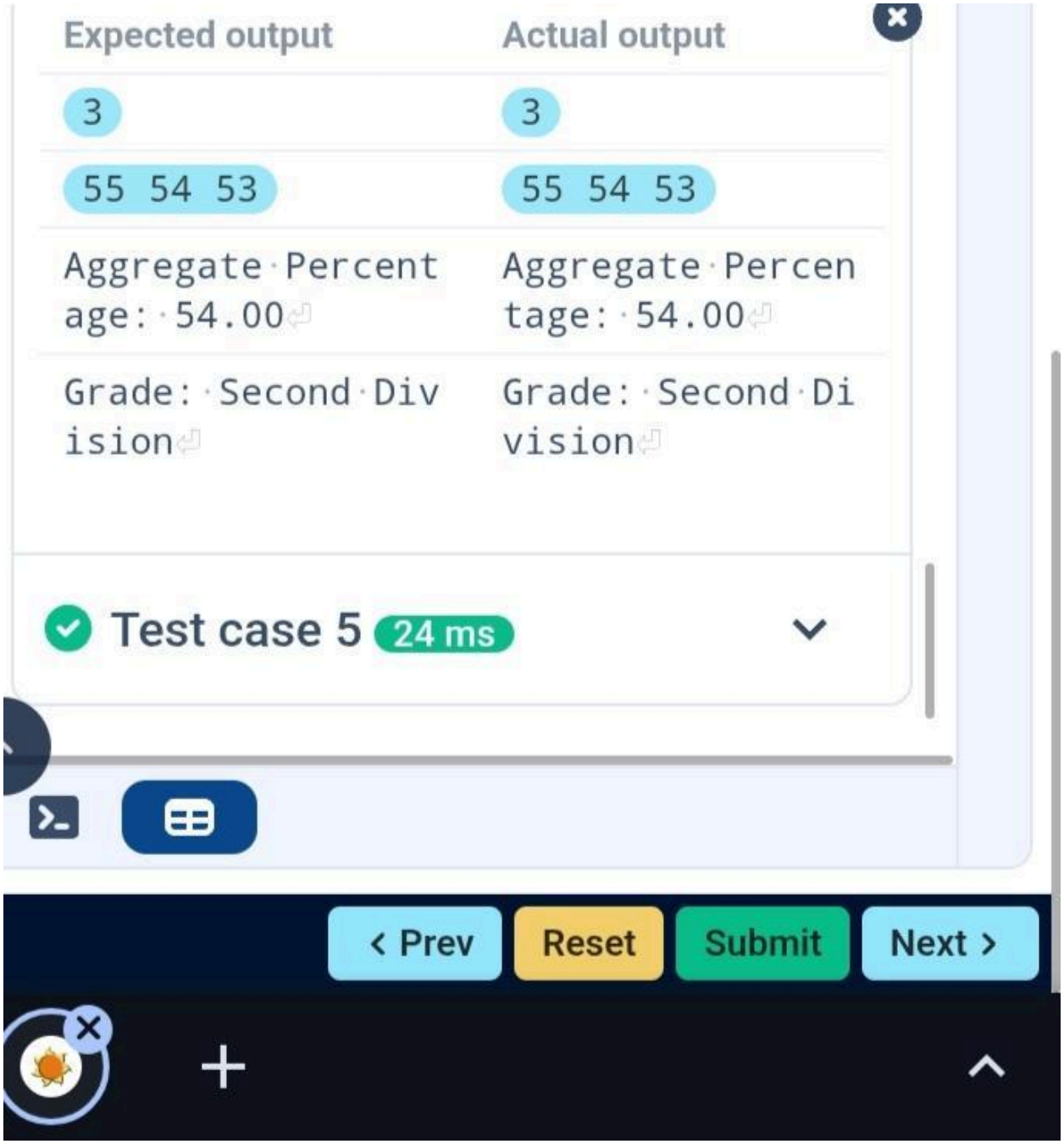
Test case 3

53 ms

Expected output	Actual output
5	5
99 85 43 97 34	99 85 43 97 34
Fail	Fail

Test case 4 31 ms

‹ Prev Reset Submit Next ›



1.2.2. Fibonacci Series using Recursive Function

Write a Python program that uses recursion to print the first n terms of the Fibonacci series.

Input Format:

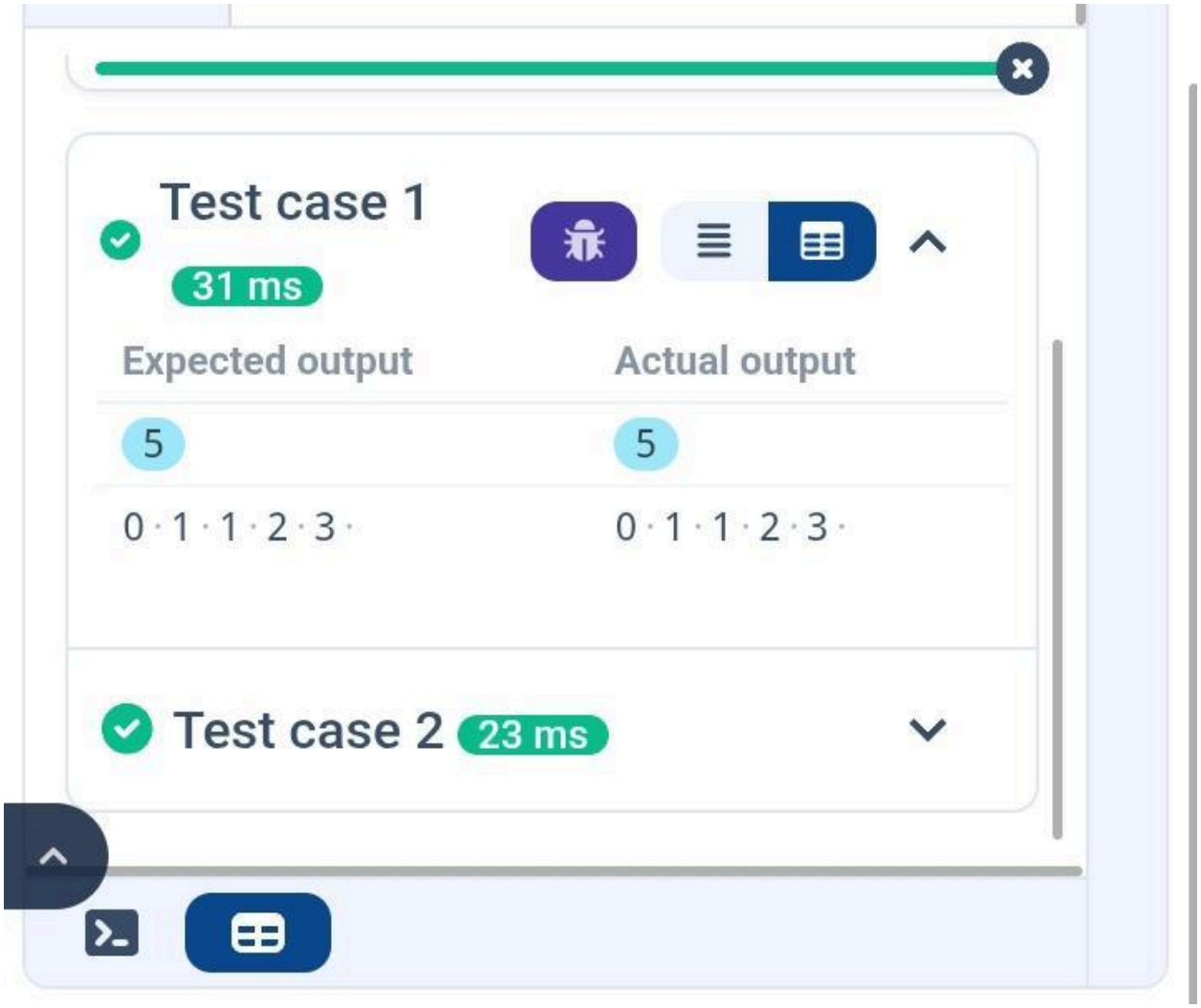
A single integer n representing the number of terms of generate.

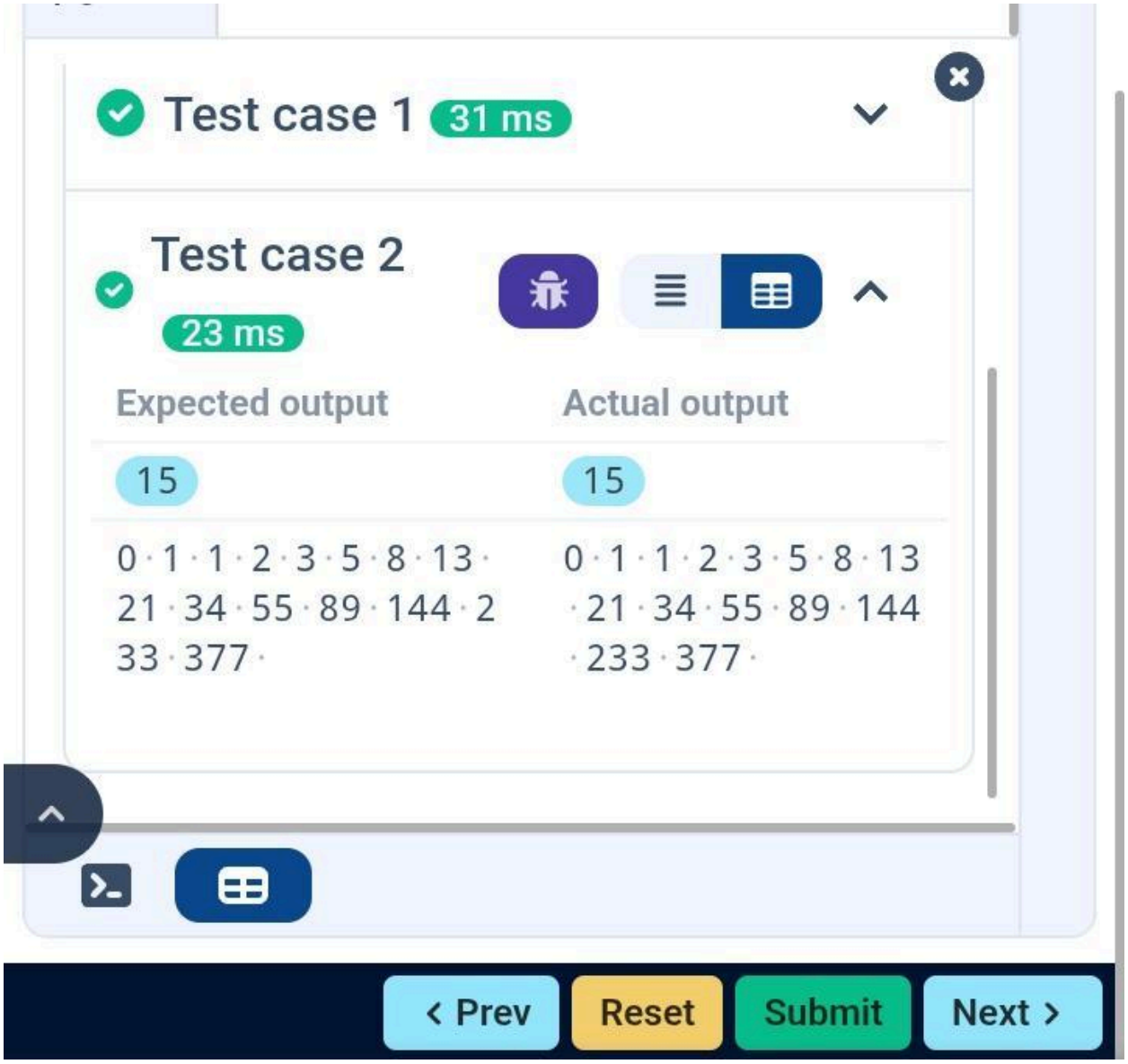
Output Format:

A single line containing the first n terms of the Fibonacci sequence, separated by spaces.

```
def fibonacci(n):
```

```
if n = 2:
return n-1
else:
return fibonacci(n-1) + fibonacci(n-2)
n = int(input())
for i in range(1, n + 1):
print(fibonacci(i), end=" ")
```


Test case 1
31 ms
Expected output
Actual output
5
5
0 · 1 · 1 · 2 · 3 ·
0 · 1 · 1 · 2 · 3 ·
Test case 2
23 ms



1.2.3. Pattern-1

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

● The input is an integer, representing the number of rows in the pattern.

Output Format

● The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

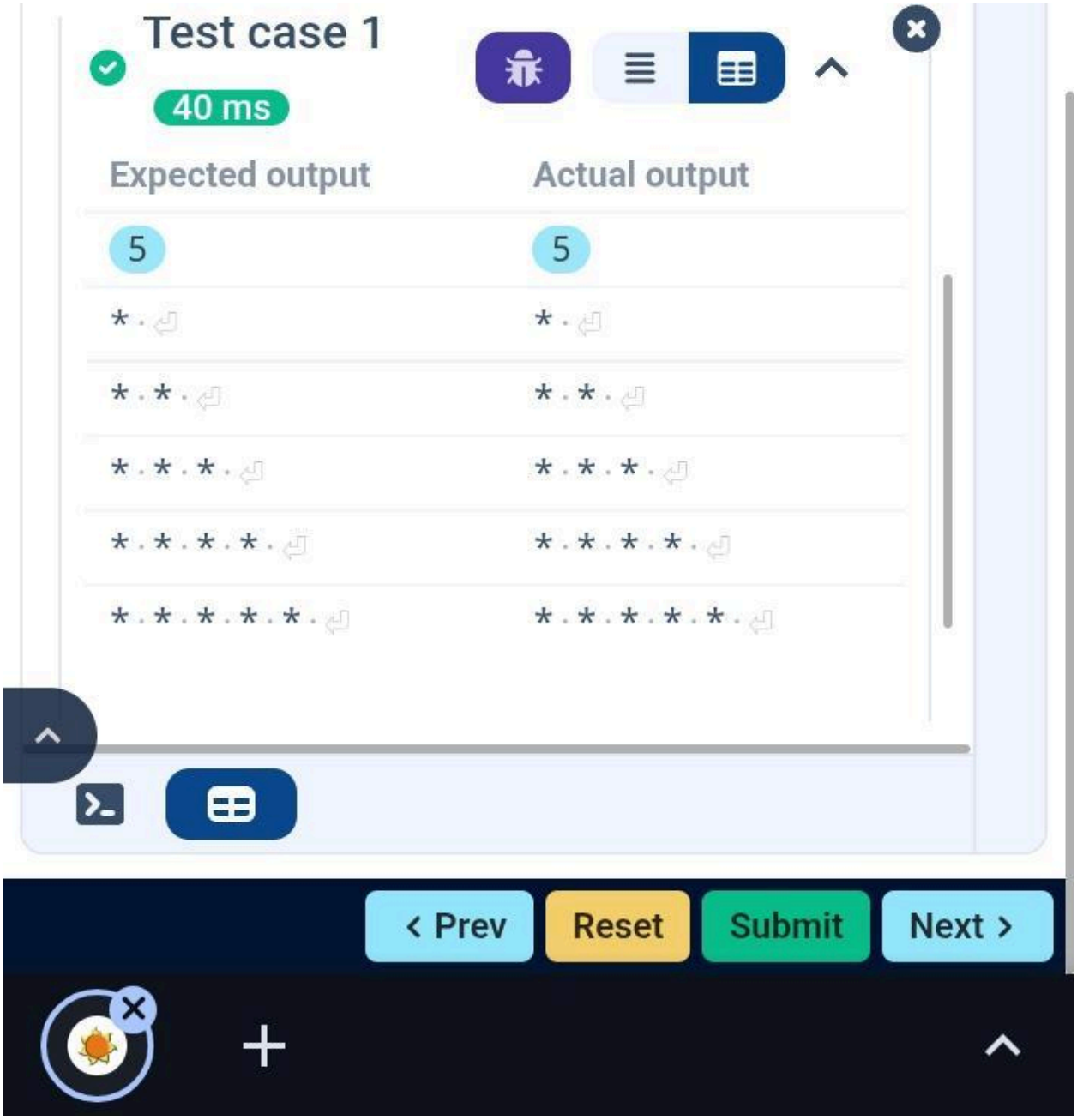
Test case 1:

4

*·⏎

..⏎
..*.⏎
..*.*.⏎

```
n = int(input())
for i in range(1, n + 1):
print("* " * i)
```



1.2.4. Pattern-2

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

● The input is an integer, representing the number of rows in the pattern.

Output Format:

● The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Note:

● Refer to the displayed test cases for the sample pattern.

Test case 1:

4

1·⏎

1·2·⏎

1·2·3·⏎

1·2·3·4·⏎

```
n = int(input())
for i in range(1, n + 1):
for j in range(1, i + 1):
print(j, end=" ")
print("")
```

Test case 1

40 ms

Expected output	Actual output
5	5
1 · ⏎	1 · ⏎
1 · 2 · ⏎	1 · 2 · ⏎
1 · 2 · 3 · ⏎	1 · 2 · 3 · ⏎
1 · 2 · 3 · 4 · ⏎	1 · 2 · 3 · 4 · ⏎
1 · 2 · 3 · 4 · 5 · ⏎	1 · 2 · 3 · 4 · 5 · ⏎

< Prev Reset Submit Next >